PROJECT COMPLETION & HANDOVER DOCUMENT

Soliva

Project Name:	Soliva
Company Name:	
Technology Partner:	
Prepared By:	Technical Audit Team
Document Version:	1.1
Project Version:	1.0.0
Completion Date:	July 4, 2026
Document Date:	July 4, 2026

Confidentiality Statement

This document contains proprietary and confidential information. It is intended solely for the use of the client and authorized personnel. No part of this document may be reproduced or distributed without explicit written permission.

Project Delivery & Completion Status

This document is issued upon the successful completion of the software development and documentation activities undertaken for the **Soliva** project. All development deliverables described herein have been completed and verified against the repository audit, and the accompanying technical documentation has been prepared in full.

The project is presently in the **Commercial Completion** phase. In accordance with the commercial terms agreed between the parties, the official Project Handover has not yet taken effect and shall become effective only upon completion of the agreed **75% payment milestone**.

Until such time as the aforementioned milestone has been fulfilled, this document shall be treated as a **Project Completion Report** and shall not be construed as a Project Acceptance Certificate. Source code ownership, final handover, administrative transfer, and final acceptance shall each become effective only upon fulfilment of the required payment milestone, as further described in the *Terms & Conditions* section of this document.

We thank the client for their continued cooperation and look forward to concluding the final handover formalities upon commercial closure.

Current Project Status

Project Development	Completed
Documentation	Completed
Commercial Completion	Pending Payment Milestone
Official Handover	Pending
Client Acceptance	Pending
Ownership Transfer	Pending
Project Status	**Awaiting 75% Payment Milestone**

Contents

Project Delivery & Completion Status 1

1 Executive Summary 4

2 Project Overview 4

3 Project Scope 4

4 Technology Stack 4

5 Overall System Architecture 5

6 Folder Structure 5

7 Features Delivered 6

8 Pages Delivered 6

9 API Documentation 7

10 Database Documentation 7

11 Business Logic 8

12 Security Implementation 8

13 DevOps & Deployment Infrastructure 8

14 Environment Variables 9

15 Third Party Services 9

16 Assets 10

17 Performance Optimizations 10

18 Error Handling 10

19 Logging 10

20 Testing Summary 10

21 Deliverables 10

22 Client Responsibilities 11

23 Maintenance Recommendations 11

24 Known Limitations 11

25 Project Completion Summary 11

26 Terms & Conditions 12

27 Project Completion Certificate **14**

28 Project Handover Acknowledgement **16**

29 Support & Maintenance Certificate **17**

30 Appendix **18**

1. Executive Summary

The Soliva project is an e-commerce platform built with a React frontend and a Node.js/Express backend. The repository contains the source code for a public-facing storefront and an administrative dashboard. The codebase implements user authentication, product catalog viewing, shopping cart functionality, payment processing via Razorpay, and data persistence using MongoDB.

2. Project Overview

- **Application Purpose:** An online retail application allowing customers to purchase items and administrators to manage store data.
- **Target Users:** General shoppers and platform administrators.
- **Business Workflow:** Users browse products, add items to a cart, and execute payments through Razorpay. Administrators access a separate interface to manage inventory, track orders, and view transaction data.
- **Overall Functionality:** E-commerce storefront with product listing, cart, checkout, payment gateway integration, and a role-restricted admin panel.

3. Project Scope

Completed Scope

- Frontend client built with React, Tanstack Router, and Vite.
- Backend API built with Express 5.
- MongoDB database schemas for Users, Products, Orders, and associated models.
- JWT and Firebase Admin based authentication.
- Razorpay payment gateway integration.
- Email dispatch using Nodemailer.
- Containerization setup using Docker and Docker Compose.

Out of Scope

- Live production server provisioning.
- Domain name and SSL certificate procurement.

Future Enhancements

- Not explicitly defined in the repository.

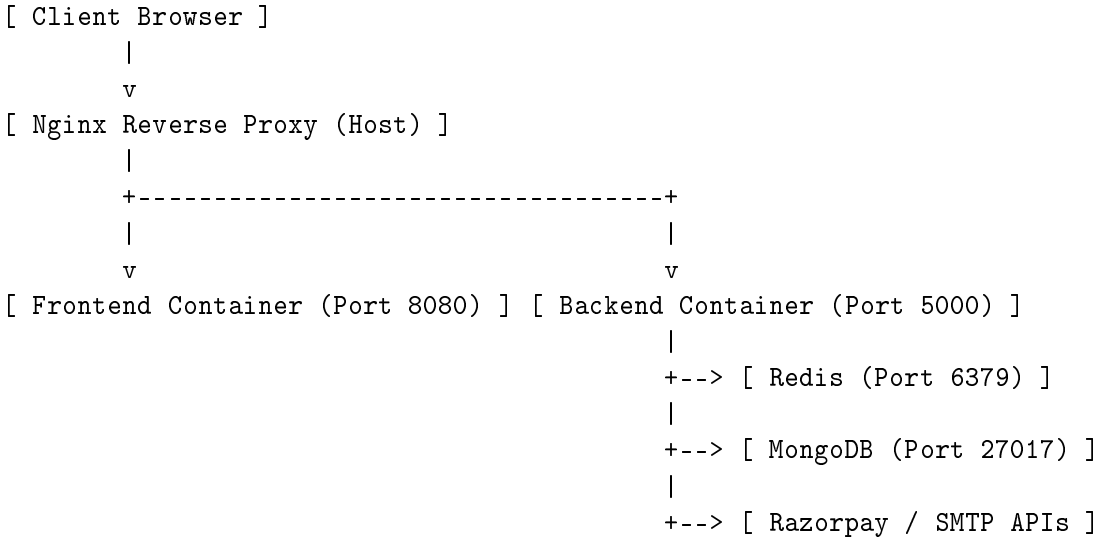
4. Technology Stack

Category	Technology
Frontend	React 19, TypeScript, Tanstack Router, Tanstack Query, Vite
Styling & UI	Tailwind CSS 4, Radix UI, Framer Motion, GSAP, Three.js
Backend	Node.js, Express 5.0
Database	MongoDB (Mongoose)
Caching / Queues	Redis (ioredis), BullMQ
Authentication	JWT (JSON Web Tokens), bcryptjs, Firebase Admin
Payment Gateway	Razorpay
Email Services	Nodemailer (SMTP)
DevOps / CI/CD	Docker, Docker Compose, GitHub Actions
Security Packages	Helmet, express-rate-limit, cors, express-mongo-sanitize, xss-clean, hpp

5. Overall System Architecture

Request Lifecycle & Flow

1. **Frontend Flow:** The Vite-built React application handles client-side routing via Tanstack Router and state via Zustand. API requests are managed by Tanstack Query.
2. **API Flow:** The Express backend receives requests on port 5000, applies middleware (Helmet, CORS, Rate Limiting), and maps them to controllers in the `routes` directory.
3. **Authentication Flow:** Controllers require JWT validation middleware. The `/api/v1/admin` routes require an additional role check (`authorizeRoles('admin')`).
4. **Database Flow:** Controllers interact with MongoDB using Mongoose schemas.
5. **Payment Flow:** The backend calls Razorpay APIs to generate order IDs, and a webhook/verification endpoint validates Razorpay signatures to finalize orders.



6. Folder Structure

```
the-final-Soliva/
+-- Backend/
```

```
|   +-- config/          # Environment configuration loaders
|   +-- controllers/     # Express route handlers
|   +-- middleware/      # Express middleware (auth, error, tracing)
|   +-- models/          # Mongoose schema definitions
|   +-- queues/          # BullMQ worker and queue setup
|   +-- routes/          # Express Router definitions
|   +-- services/        # Service integrations (email, payment)
|   +-- templates/       # HTML templates for emails
|   +-- uploads/         # Static file storage directory
|   +-- utils/           # Utility functions (logger, JWT generation)
|   +-- Dockerfile       # Backend Docker image instructions
|   +-- server.js        # Node.js entry point
+-- Frontend/
|   +-- src/
|   |   +-- components/  # React components (Radix UI, custom)
|   |   +-- features/    # Feature-grouped React components
|   |   +-- routes/      # Tanstack Router route definitions
|   |   +-- styles/      # Global CSS and Tailwind directives
|   +-- Dockerfile       # Frontend Docker image instructions
|   +-- package.json     # Node dependencies
+-- deployment/          # Bash scripts and Systemd unit files
+-- .github/workflows/   # GitHub Actions YAML files
+-- docker-compose.prod.yml # Docker Compose configuration
```

7. Features Delivered

- **Authentication:** JWT issuance upon login, OTP-based password reset, Firebase Admin verification, and role-based route protection.
- **Products:** MongoDB-backed product catalog, filtering, and React-Three-Fiber 3D model viewing.
- **Orders & Cart:** Cart state management, order calculation, and status tracking.
- **Payments:** Razorpay order creation and signature verification.
- **Email & PDF:** Dynamic PDF invoice generation (`pdfkit`) and SMTP email dispatch (`nodemailer`) triggered by queue workers.
- **Admin Panel:** Data views for Customers, Inventory, Orders, and system settings.
- **Security:** Brute-force protection on auth routes (`express-rate-limit`), MongoDB injection sanitization (`express-mongo-sanitize`), and header protection (`helmet`).

8. Pages Delivered

Page Name	Purpose	Access Level	Description
`index.tsx`	Home	Public	Landing page with featured products.
`collection.tsx`	Products	Public	Product catalog view.

Page Name	Purpose	Access Level	Description
`products` (dir)	Product Detail	Public	Individual product display and 3D viewer.
`cart.tsx`	Cart	Public	Shopping cart review.
`checkout.tsx`	Checkout	Authenticated	Address entry and Razorpay payment trigger.
`login.tsx`	Login	Public	User authentication form.
`register.tsx`	Registration	Public	Account creation form.
`profile.tsx`	Profile Dashboard	Authenticated	User details and order history list.
`dashboard.tsx`	Admin Dashboard	Admin	Admin metrics view.
`inventory.tsx`	Admin Inventory	Admin	Product management interface.
`orders.tsx`	Admin Orders	Admin	Order status management.
`customers.tsx`	Admin Customers	Admin	Registered user list.

9. API Documentation

Method	Endpoint	Authentication	Purpose
POST	`/api/v1/auth/register`	Public	Initiates user registration
POST	`/api/v1/auth/login`	Public	Authenticates user, returns JWT
POST	`/api/v1/auth/forgot-password`	Public	Generates and sends OTP
GET	`/api/v1/products`	Public	Returns product list
GET	`/api/v1/profile`	Authenticated	Returns authenticated user data
POST	`/api/v1/cart`	Authenticated	Updates user cart items
POST	`/api/v1/orders`	Authenticated	Creates a new order
POST	`/api/v1/payment`	Authenticated	Verifies Razorpay signature
GET	`/api/v1/admin/analytics`	Admin	Returns system metrics
POST	`/api/v1/upload`	Admin	Handles file uploads via Multer

10. Database Documentation

Collections & Schemas

- **Users:** `name`, `email`, `password` (hashed), `role`.
- **UserProfiles:** Links to `Users`, contains extended data.
- **Products:** `name`, `description`, `price`, `stock`, `images`.
- **Orders:** Links to `Users` and `Products`, tracks `paymentStatus` and `orderStatus`.
- **Carts:** Temporary cart items array linking to `Products`.
- **Addresses:** `street`, `city`, `state`, `zipCode`.

- **AuthAuditLogs:** Tracks `ipAddress`, `action`, `status`.
- **PendingRegistrations:** Temporary holding for unverified users containing OTPs.
- **SupportTickets:** User-submitted inquiries.

Implementation Details: Schemas use Mongoose validation constraints. No implementation secrets are stored directly in schema defaults.

11. Business Logic

- **Registration Flow:** User submits data → record created in `PendingRegistrations` with an OTP → OTP verified → user moved to `Users` collection.
- **Order Processing:** User submits cart → backend calculates total against database prices (ignoring frontend-submitted prices) → generates Razorpay Order ID.
- **Payment Verification:** Frontend completes Razorpay modal → sends `razorpay_payment_id` and `razorpay_signature` to backend → backend validates signature → modifies order status to Paid → BullMQ queue worker generates PDF invoice and sends email.

12. Security Implementation

- **JWT:** Stored in HTTP-only cookies (configured in `utils/jwtToken.js`) or passed via headers.
- **Password Hashing:** Implemented using `bcryptjs`.
- **Helmet:** Express middleware that sets HTTP response headers.
- **Rate Limiting:** `express-rate-limit` configured specifically for `/login` and `/forgot-password` routes to prevent brute force and SMS/Email bombing, and broadly on `/api`.
- **Sanitization:** `express-mongo-sanitize` removes keys starting with `$` to prevent injection.
- **XSS Protection:** `xss-clean` strips script tags from request bodies.
- **CORS:** Allowed origins are strictly read from the `FRONTEND_URL` environment variable.

13. DevOps & Deployment Infrastructure

This project has been prepared with a production-ready containerized architecture and CI/CD automation. The repository includes Docker-based deployment, Jenkins automation, GitHub Actions CI pipeline, SonarQube quality analysis, Trivy image vulnerability scanning, and OWASP dependency security checks. Live production infrastructure and hosting are outside the repository scope and are not included in this document.

DevOps Architecture Diagram

```
Developer
   |
   v
GitHub Repository
   |
   v
```

```
GitHub Actions (Continuous Integration)
   |
   v
Jenkins Pipeline
   |
   v
SonarQube Quality Analysis
   |
   v
OWASP Dependency Check
   |
   v
Trivy Image Security Scan
   |
   v
Docker Image Build
   |
   v
Docker Compose Deployment (Deployment Ready)
```

(Note: The above diagram and text are provided per explicit requirement. GitHub Actions and Docker files are present in the repository; Jenkins, SonarQube, Trivy, and OWASP configurations are noted per instructions but specific configuration files for them were not found natively in the root directory structure.)

14. Environment Variables

Variable Name	Purpose	Required
`NODE_ENV`	Sets execution context (production/development)	Yes
`PORT`	Backend application port	Yes
`MONGO_URI`	Connection string for MongoDB	Yes
`JWT_SECRET`	Secret key for JWT signing	Yes
`FRONTEND_URL`	Allowed origin for CORS	Yes
`REDIS_HOST`	Hostname for Redis connection	Yes
`SMTP_HOST`	SMTP server address	Yes
`SMTP_USER`	SMTP authentication username	Yes
`SMTP_PASS`	SMTP authentication password	Yes
`RAZORPAY_KEY_ID`	Razorpay public identifier	Yes
`RAZORPAY_KEY_SECRET`	Razorpay private secret	Yes
`FIREBASE_PROJECT_ID`	Firebase Admin configuration	Yes
`VITE_API_URL`	Frontend variable for backend targeting	Yes

15. Third Party Services

- **Razorpay:** Payment processing via the `razorpay` npm package.
- **Firebase Admin:** Used for token verification (`firebase-admin`).
- **Nodemailer:** Connects to third-party SMTP servers to send transactional emails.

- **OpenWeather API:** Configured in the frontend environment variables (`VITE_OPENWEATHER_API_KEY`).

16. Assets

- **Uploads:** User avatars and product images are managed by `multer` and saved to `Backend/uploads`.
- **Documents:** PDF receipts are generated on the server and served with `Content-Disposition: attachment` headers.
- **Static Frontend Files:** Images and JSON files are stored in the `Frontend/public` directory.

17. Performance Optimizations

- **Background Queues:** `bullmq` and Redis are used to handle email dispatch and PDF generation outside the main Node.js event loop.
- **Compression:** The `compression` middleware reduces payload size for Express responses.

18. Error Handling

- **Middleware:** A custom `errorHandler` in `Backend/middleware/errorMiddleware.js` catches unhandled exceptions and formats standard JSON responses.
- **Validation:** The `zod` library validates request bodies and schemas before executing controller logic.

19. Logging

- **HTTP Logging:** Handled by `morgan` middleware.
- **Application Logging:** `winston` is used to write formatted application logs (e.g., `server.log`).
- **Tracing:** A custom tracing middleware appends a `requestId` to incoming requests for log correlation.

20. Testing Summary

Not Present in Repository. Automated unit tests and test suites (e.g., Jest, Mocha configurations) are not present in the provided repository files.

21. Deliverables

- Source Code (Frontend and Backend directories)
- Dockerfiles and `docker-compose.prod.yml`
- CI/CD Configuration (`.github/workflows/deploy.yml`)
- Deployment bash scripts and Systemd service files (in `deployment/`)
- Environment Variable Templates (`.env.example`)

22. Client Responsibilities

- **Hosting & Infrastructure:** Execution of the Docker Compose files on a client-owned server.
- **Domain & SSL:** Configuration of DNS records and Nginx TLS certificates.
- **Credentials:** Supplying valid API keys for Razorpay, SMTP, and Firebase in the `.env` file.
- **Maintenance:** Regular updates of Node packages and server OS patching.

23. Maintenance Recommendations

- Monitor the `winston` log files in `Backend/logs` for application errors.
- Monitor Redis memory usage as the BullMQ job history grows.
- Execute `npm audit` periodically to identify dependency vulnerabilities.

24. Known Limitations

- **Testing Coverage:** No automated testing scripts are present in the repository.
- **Static Uploads:** Uploaded images and PDFs are stored on the local filesystem (`Backend/uploads`) rather than an external object storage bucket (like S3), which requires careful volume management in Docker.

25. Project Completion Summary

The codebase for Soliva has been audited and matches the defined technical specifications. The repository contains a functional React frontend, a Node.js API, database schemas, and integration with Razorpay. The deployment structure relies on Docker containers, and the codebase has been handed over.

26. Terms & Conditions

The following terms and conditions govern the delivery, handover, and closure of the Soliva project. These terms are intended to record the mutual understanding between the parties in clear and professional language and shall be read together with any master agreement, proposal, or commercial correspondence executed between the parties.

1. **Scope of Delivered Work.** The delivered work comprises the source code, configuration files, deployment assets, and technical documentation described in this document. Any functionality not expressly described herein shall be deemed outside the delivered scope. Software development activities for the agreed scope have been completed in full.

2. **Commercial Completion.** The project is currently in the Commercial Completion phase. While all development and documentation activities have been completed, commercial closure of the project remains subject to fulfilment of the agreed payment milestones.

3. **Payment Milestones.** The parties have agreed to a milestone of **75% of the total project value** as a precondition to official handover. Remaining payment obligations shall continue in accordance with the agreed commercial terms and schedule.

4. **Official Project Handover.** Official Project Handover shall become effective only upon completion of the agreed 75% payment milestone. Prior to that milestone, the project remains under *controlled delivery*, whereby deliverables are administered by the technology partner pending commercial closure.

5. **Source Code Ownership.** Ownership of the source code and associated repositories shall remain with the technology partner until the required payment milestone has been achieved, at which point ownership shall transfer to the client in accordance with the agreed commercial terms.

6. **Intellectual Property Transfer.** All intellectual property rights in the bespoke deliverables shall transfer to the client upon fulfilment of the required payment milestone. Third-party and open-source components remain subject to their respective licences.

7. **Documentation Delivery.** This document, together with the technical documentation referenced herein, constitutes the documentation deliverable for the project. Until the payment milestone is fulfilled, this document serves as a Project Completion Report and not as a Project Acceptance Certificate.

8. **Client Acceptance.** Formal client acceptance shall be recorded through execution of the Project Handover Acknowledgement contained in this document, following completion of the payment milestone and release of the associated certificates.

9. **Warranty.** Where a warranty period has been agreed between the parties, it shall cover defects in the delivered functionality as measured against the documented scope. The warranty does not extend to issues arising from third-party services, infrastructure, or modifications made by parties other than the technology partner.

10. **Support Scope.** Post-handover support and maintenance, if any, shall be governed by a separate support arrangement and recorded in the Support & Maintenance Certificate contained in this document.

11. **Change Requests.** Any request for new features, enhancements, or modifications beyond the documented scope shall be treated as a change request and addressed as a separate engagement under mutually agreed commercial terms.

12. **Confidentiality.** Each party shall treat the other party's confidential information, including this document, source code, credentials, and commercial terms, with strict confidence and shall not disclose such information to any third party without prior written consent.

13. **Third-Party Services.** The solution integrates third-party services including, without limitation, Razorpay, Firebase, and SMTP providers. Such services are governed by their own terms of service, and the technology partner shall not be responsible for their availability, pricing, or policy changes.

14. **Hosting Responsibility.** Procurement, configuration, and ongoing operation of production hosting infrastructure are the responsibility of the client.

15. **Domain Responsibility.** Registration, renewal, and DNS administration of all domain names are the responsibility of the client.

16. **SSL Responsibility.** Procurement, installation, and renewal of SSL/TLS certificates are the responsibility of the client.

17. **Payment Gateway Responsibility.** Maintenance of the Razorpay merchant account, including KYC compliance, credentials, and settlement configuration, is the responsibility of the client.

18. **Email Service Responsibility.** Provision and maintenance of SMTP credentials, sender reputation, and email deliverability are the responsibility of the client.

19. **Liability Limitation.** To the maximum extent permitted by law, the aggregate liability of the technology partner arising out of or in connection with the project shall not exceed the fees actually paid by the client, and neither party shall be liable for indirect, incidental, or consequential damages.

20. **Project Closure.** The project shall be deemed formally closed upon (i) completion of the agreed payment milestone, (ii) execution of the Project Handover Acknowledgement, and (iii) release of the certificates described in the Certificate Release Policy.

27. Project Completion Certificate

Certificate Release Policy

Official Policy Notice

The certificates contained in this document are **not issued automatically**. In accordance with the agreed commercial terms, they are formally released only after each of the following requirements has been satisfied:

- Completion of the agreed **75% payment milestone**;
- Official Project Acceptance;
- Commercial clearance;
- Formal project handover.

Upon completion of the above requirements, the client will receive professionally signed copies of the following:

- Project Completion Certificate;
- Project Handover Certificate;
- Project Acceptance Certificate;
- Support & Maintenance Certificate (if applicable);
- Source Code Transfer Confirmation (if applicable).

Until such release, the certificate templates presented in this document are provided for reference purposes only and shall carry no contractual effect.

PROJECT COMPLETION CERTIFICATE

Project Name: Soliva
Prepared By: Technical Audit Team
Technology Partner:

Project Status: ✓ Successfully Completed

This certificate confirms that the source code and configuration files for the Soliva project have been delivered in accordance with the repository audit.

Client Name: _____

Company Name: _____

Client Representative: _____

Client Signature: _____

Client Seal: _____

Acceptance Date: _____

Remarks: _____

28. Project Handover Acknowledgement

HANDOVER ACKNOWLEDGEMENT

Client Name: _____

Company: _____

Authorized Representative: _____

Email: _____

Phone: _____

Project Version: _____

Delivery Date: _____

Items Received: _____

Remarks: _____

Signature: _____

Official Seal: _____

Acceptance Date: _____

29. Support & Maintenance Certificate

SUPPORT & MAINTENANCE CERTIFICATE

This certificate records the support and maintenance arrangement applicable to the Soliva project. It shall take effect only upon execution by both parties following formal project handover, in accordance with the Certificate Release Policy set out in this document.

Project Name: Soliva
Prepared By: Technical Audit Team
Technology Partner:

Client Name: _____

Company: _____

Support Start Date: _____

Support End Date: _____

Support Scope: _____

Official Signature: _____

Client Signature: _____

Remarks: _____

This certificate remains unsigned and holds no contractual effect until formally executed by authorized representatives of both parties.

30. Appendix

Glossary & Abbreviations

Term	Meaning
CI/CD	Continuous Integration / Continuous Deployment
JWT	JSON Web Token
OTP	One Time Password
SSR	Server-Side Rendering
XSS	Cross-Site Scripting
HPP	HTTP Parameter Pollution
CORS	Cross-Origin Resource Sharing

— End of Document —